Full Stack Development with MERN - Project Documentation

1. Introduction

Date	**29 September 2026**
Team id	**SWTID-2026-7867**
Team Leader	**M.Mohammed Firdouse**
Team Members	**S.Subash,S.Mohamed Ashik, H.Abdul Arsath**
Project name	**Import Data using Transform Maps (Spreadsheet)**

2. Project Overview

Purpose: To develop an automation & data management tool for transforming, mapping, and securely importing data coming from Spreadsheets (Excel/CSV) into a database (MongoDB) in a MERN stack application.

Features:

- Facility to upload spreadsheet files (CSV/XLSX).
- Option to set column mapping and transformation rules.
- Data validation and error handling.
- Dashboard to track import history and logs.

3. Architecture

- *Frontend:* Built using http://React.js to create a dynamic file upload interface, mapping UI, and status dashboard.

- *Backend:* Built using http://Node.js and http://Express.js to handle file parsing (xlsx/csv parser), transform logic, and REST APIs.

- *Database:* Schemas for users, uploaded files, transformation rules, and imported data are designed in MongoDB.

4. Setup Instructions

- *Prerequisites:* http://Node.js, npm, MongoDB local/cloud instance.

- *Installation:*

1. Clone the repository: `git clone <repository-url>`
2. Install server dependencies: `cd server && npm install`
3. Install client dependencies: `cd client && npm install`
4. Configure environment variables (.env) in the server directory (PORT, MongoDB URI).

5. Folder Structure

- *Client (React):*

  - `src/components`: FileUpload, TransformMapper, Dashboard, Navbar.

  - `src/services`: API call configurations (Axios).

  - `src/App.js`: Main router setup.

- *Server (Node.js/Express):*

  - `models`: http://User.js, http://ImportLog.js, http://DataSchema.js

  - `routes`: http://uploadRoutes.js, http://transformRoutes.js

  - `controllers`: http://uploadController.js, http://transformController.js

  - `middleware`: http://multerConfig.js (file upload handling)

6. Running the Application

- *Backend Server:* `cd server` and `npm start` (or `npm run dev`)

- *Frontend Client:* `cd client` and `npm start`

7. API Documentation

- `POST /api/upload`: An endpoint that sends the spreadsheet file to the server and parses it temporarily.

- `POST /api/transform`: An endpoint that maps the columns and inserts the data into the database.

- `GET /api/logs`: Fetches the data import history done so far.

8. Authentication

- User authentication is handled using JSON Web Tokens (JWT). Bearer tokens must be sent for secure endpoints.

9. User Interface

- The UI includes a drag-and-drop file upload zone, interactive column mapper dropdowns, and import progress indicators.

10. Testing

- *Backend Testing:* API endpoints can be tested using Jest and Supertest.

- *Frontend Testing:* Component rendering can be tested using React Testing Library.

11. Screenshots or Demo

- [Add Frontend UI screenshots and demo link here]

12. Known Issues

- Memory optimization may be required when importing large-scale (> 50,000 rows) spreadsheet files.

13. Future Enhancements

- Facility to import large files asynchronously using background queue processing (BullMQ).

- Supporting JSON, XML formats in addition to Excel/CSV.